



Week 8

✓ CPU Scheduling

- Major Type of CPU Scheduler
- Scheduling Algorithm Optimization Criteria
- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Shortest Remaining Time First (SRTF)
- Priority Scheduling
- Round Robin (RR)
- Multi level queue
- Multi level Feedback queue
- Java Thread Scheduling

| Content

✓ CPU Scheduling

- the way that CPU choose process or thread to run
- At the time, there are many process or thread waiting in the ready queue and CPU has to choose one
 - maybe simplest algorithm → first come, first serve (not really efficient)
 - OS may use some other algorithm



Major Type of CPU Scheduler

CPU Scheduler

- Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them
- **nonpreemptive**
- **preemptive**
- Nonpreemptive
 - after OS choose a process or thread to run, OS doesn't have a way to stop a process
 - the process must willing to finish using the CPU itself, maybe by execute the last statement, or maybe it said i gonna go out for a while(wait for i/o).
 - when OS give process or thread a chance to run → OS can't interrupt that process.
- Preemptive
 - OS can interrupt the process



Scheduling Algorithm Optimization Criteria

- no matter the algorithm that OS used, OS need to make sure that this thing gonna happen when OS used the algorithm, some of them conflict to each other.

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time
- Max CPU utilization
 - when OS use algorithm, OS has to make sure that it has to get the max CPU utilization.

- CPU should be busy, CPU should not wait idle.
- we need to make sure that CPU work (always busy)
- Max throughput
 - throughput → number of process that finish at a time.
 - ex, process can finish 20 per second is better than 10 per second.
- Min turnaround time
 - turn around time → time that count from the time that process arrive the system until the process finish.
- Min waiting time
 - waiting time → the time that the process wait in the ready queue (waiting time is a part of turnaround time).
- Min response time
 - response time → the time that when process first response to the user.
 - if you ask the process to do something → first response to you.

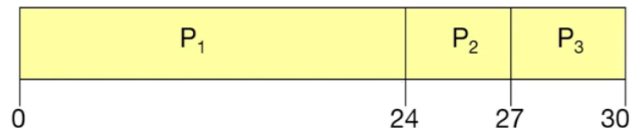
If you want better response time → turn around time get worse. (turn around time and response time always conflict to each other.)

First come First Serve

- Non-preemptive (in this, let's assume → when terminate)

First-Come, First-Served (FCFS) Scheduling

Process	Arrival Time	Burst Time
P_1	0.0	24
P_2	2.0	3
P_3	4.0	3



- Waiting time for $P_1 = 0$; $P_2 = 22$; $P_3 = 23$
- Average waiting time: $(0 + 22 + 23)/3 = 15$

Arrive → ready in ready queue

- In this slide, we assume that we have 3 processes that we need to consider.
 - Process P1 arrived at time 0, and process P1 said it need 24 time unit to finish its task.
 - Process P2 arrived at time 2, and process P2 said it need to use 3 unit of time
 - Process P3 arrived at time 4, and process P3 said it need to use 3 unit of time
- To calculate → Draw the chart (necessary to calculate waiting time and turn around time)
 - Non preemptive scheduling → P1 start and OS can't interrupt.
 - P2 come first, P2 next
 - P3 next
- Waiting time (Start time- Arrival time) (wait in ready queue)
 - $P1 = 0$
 - $0 - 0 = 0$
 - $P2 = 22$
 - $24 - 2 = 22$
 - $P3 = 23$

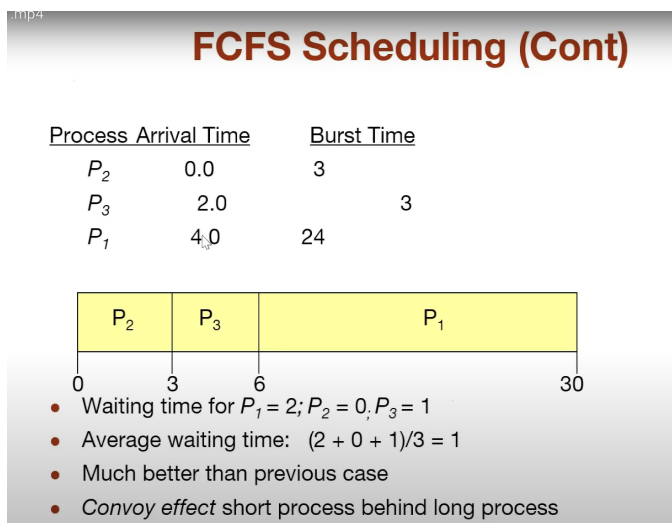
- $27 - 4 = 23$
- Average waiting time $(0+22+23)/3 = 15$
- Calculate Turn around time (arrival time until its finish)
 - $P1 = 24 - 0 = 24$
 - $P2 = 27 - 2 = 25$
 - $P3 = 30 - 4 = 26$



Relationship between waiting time and turn around time

- Waiting time = Turn around time - Burst Time
- Waiting time for P1
 - $0 = 24 - 24$
- Waiting time for P2
 - $22 = 25 - 3$
- Waiting time for P3
 - $23 = 26 - 3$

Another Example



👉 if the process come in this order, average waiting time ดีกว่ามาก
 in FCFS, the performance depends on the order of process (which one come first) ถ้าอันเวลาน้อยมาก่อน performance ดี
 [same concept of when you come to the bank]

- FCFS is easy to implement, but the performance depends on which process come first.

📌 Shortest Job First

Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
 - The difficulty is knowing the length of the next CPU request

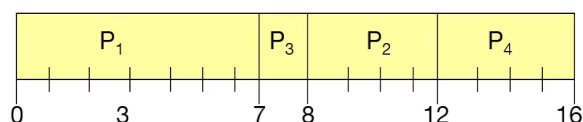
Non preemptive algorithm

- idea is at the time, OS need to choose process to run, OS choose process that use the least burst time (least time to run)

Example of SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF scheduling chart



- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

- At time 0, because only P1 is in the system and it non-preemptive → allow to run until finish

- Then p3 next, because P2, P3, P4 is waiting but P3 has the least burst time
- P2, P4 choose P2 → same burst time, but P2 arrived first
- Waiting Time (Start time - Arrival Time)
 - P1 = 0
 - 0 - 0
 - P2 = 6
 - 8 - 2
 - P3 = 3
 - 7 - 4
 - P4 = 7
 - 12 - 5
- Average Waiting Time = (0+6+3+7)/4 = 4
- Turn around Time (arrival time until it finish) (finish time - arrival time)
 - P1 = 7 - 0 = 7
 - P2 = 12 - 2 = 10
 - P3 = 8 - 4 = 4
 - P4 = 16 - 5 = 11

Shortest Remaining Time First

Shortest Remaining Time First (SRTF)

- Preemptive version of the SJF algorithm
- Processor allocated to job closest to completion
 - Current job can be preempted if newer job in READY queue has shorter time to completion
- Cannot be implemented in interactive system
 - Requires advance knowledge of the CPU time required to finish each job
- SRTF involves more overhead than SJF
 - OS monitors CPU time for all jobs in READY queue and performs context switching

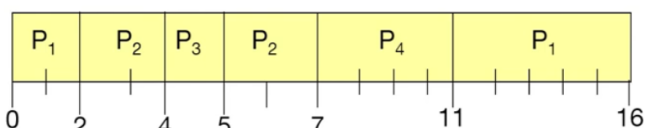
- preemptive

- Every time new process arrived the system, OS will check if the new process use less time than running process or not (q remaining time)
 - if it is, Stop running process, allow new process to run instead.

Example of SRTF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



- Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

- Find turnaround time first → then find waiting time → easier
 - Turn around time
 - $P_1 = 16 - 0 = 16$
 - $P_2 = 7 - 2 = 5$
 - $P_3 = 5 - 4 = 1$
 - $P_4 = 11 - 5 = 6$
- Then user formula → waiting time = turn around time - burst time
 - $P_1 = 16 - 7 = 9$
 - $P_2 = 5 - 4 = 1$
 - $P_3 = 1 - 1 = 0$
 - $P_4 = 6 - 4 = 2$

3 algorithms above is not practical because actually in real system, we never know burst time of each process. Each

process never announce how much time it take for them to finish.

Priority Scheduling

So why learn? in practice, we use priority

- the idea is → we can specify the priority of the process or the thread.
 - You can think that you can apply SJF or SRTF to the priority
 - You can think that the process with higher priority use less burst time, and use the same algorithm (higher priority, less burst time)

Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - nonpreemptive
- SJF is a priority scheduling where priority is the predicted next CPU burst time
- Problem $\equiv$ **Starvation** – low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process

- if we use preemptive → work just like SJF
- if we use non preemptive → work just like SRTF
- priority number is based on OS → some consider that higher number, higher priority, some consider smaller number, higher priority. (many OS use smaller number → higher priority)

Problem: Starvation means the process that arrived the system with very low priority may never have a chance to execute. process with higher priority always arrived to the system

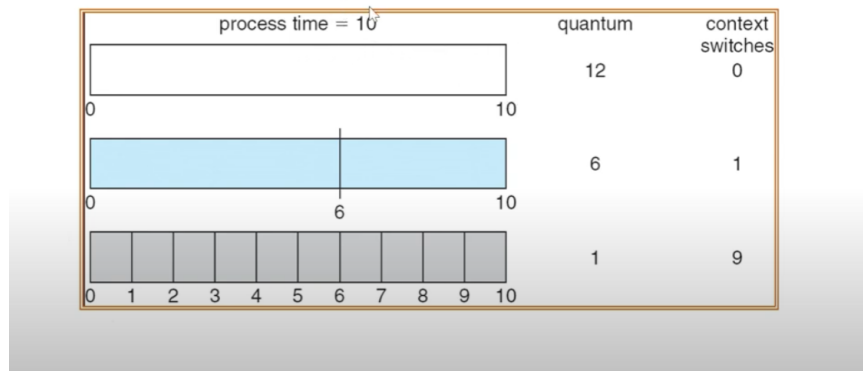
Solution: Aging means priority of process will be increased based in the time the process wait in the system. (until process has high enough priority to run)

Round Robin

Round Robin (RR)

- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
 - If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
 - Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high
-
- Same concept as time sharing (the process has the same amount of CPU time to run $\rightarrow$ if not have enough time $\rightarrow$ go wait for next turn)
 - For Round Robin,
 - Each process gets a small same about unit of cpu time (time quantum - in unit of mili second, depends on the system)
 - Each process after it use all un the time quantum and not finish yet $\rightarrow$ go to ready and wait for next turn.
 - The process swap in and out of the cpu based on the time unit
 - RR can give the system better response time and can gauruntee that each process won't wait more than $(n-1)q$ time unit
 - Ex, $Q = 10$ mili second, $M = 4$ processes
 - Wait no more than $(4 - 1) 10 = 30$ mili seconds
 - Performance depends on time quantum
 - if you set very large $\rightarrow$ become first come first serve
 - **if too small $\rightarrow$ increase number of context switch (and context switch take time)**

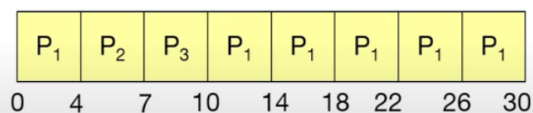
Time Quantum and Context Switch Time



Example of RR with Time Quantum = 4

Process	Arrival Time	Burst Time
P_1	0.0	24
P_2	2.0	3
P_3	4.0	3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better *response*

- Turn around time
 - $P_1 \rightarrow 30 - 0 = 30$
 - $P_2 \rightarrow 7 - 2 = 5$
 - $P_3 \rightarrow 10 - 4 = 6$
- Waiting time
 - $P_1 \rightarrow 30 - 24 = 6$
 - $P_2 \rightarrow 5 - 3 = 2$
 - $P_3 \rightarrow 6 - 3 = 3$

Multilevel Queue

(idea that make system has better performance)

Multilevel Queue

- Ready queue is partitioned into separate queues:
foreground (interactive)
background (batch)
- Each queue has its own scheduling algorithm
 - foreground – RR
 - background – FCFS
- Scheduling must be done between the queues
 - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
 - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR 20% to background in FCFS
- in some OS, os may divide ready queue in 2 levels, and each level can has it own algorithm (can use different algorithm)
- 1 example is hybrid OS
 - foreground queue can use RR
 - background queue can use FCFS
 - (this just example, actually, it can use any, it can also divide into more than 2 queue)
- Divide ready queue in 2 level also increase the complexity of a task that OS need to perform.
- If OS use fixed priority scheduling like foreground always more importance than background , if there process in foreground, we never execute the process in background → the process in background may get starvation. (foreground process always submit to the system.)
 - Solution → Time Slice (OS need to do more complex task to balance between more than 1 ready queue)

Multilevel Feedback Queue

(some os also use more advanced feature called multilevel feedback queue)

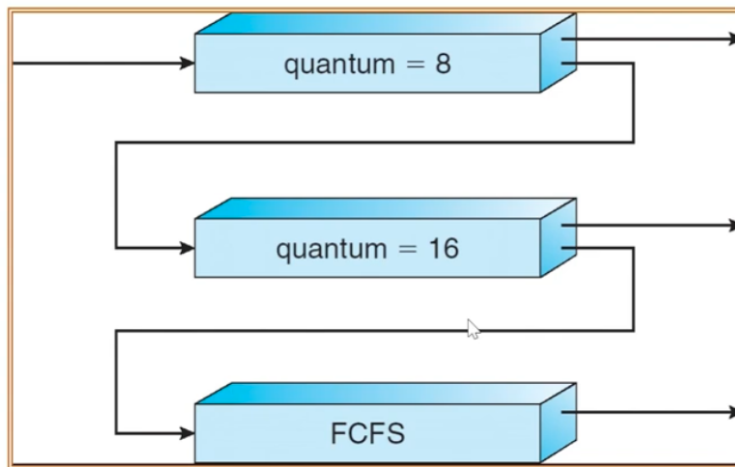
Multilevel Feedback Queue

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service
- Major different between multilevel feedback queue and multi level queue
 - for multi level queue → each process never change its queue
 - for multilevel feedback queue → each process can change the queue

Example of Multilevel Feedback Queue

- Three queues:
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS
- Scheduling
 - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1 .
 - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2 .
- OS said that all process start in Q_0 and if it can't finish in 8 mili second → go down to Q_1 (and if it can't finish in 16) → go to Q_2 .

Multilevel Feedback Queues



for this one, this system give more benefit to the process that use less than 8 mili seconds

Multilevel Feedback Queue

- A process can move between the various queues; aging can be implemented this way
 - Multilevel-feedback-queue scheduler defined by the following parameters:
 - number of queues
 - scheduling algorithms for each queue
 - method used to determine when to upgrade a process
 - method used to determine when to demote a process
 - method used to determine which queue a process will enter when that process needs service
-
- if the process go from higher priority to lower priority → demote
 - if the process go from lower priority to higher priority → upgrade

The process can go up and down depends on OS whether it give chance to process or not.

Java Thread Scheduling

Java Thread Scheduling

- Java specification does not specify the standard scheduling. It depends on the implementation of each JVM.
- **Do not rely on the scheduling algorithm for the correctness of your program.**
- Actually, JVM always use the thread library on host OS, this mean when java said it write once, run anywhere → it may not apply to this thread programming, thread may behave differently based on the OS that JVM run.
- when you write multi threaded program → you must think about this one.
 - you don't know which algorithm your thread program will be used.

Java Threads Scheduling (Cont.)

- Normally, Java uses Preemptive and priority based scheduling algorithm.
 - All Java threads have a priority. The thread with the highest priority is chosen first.
 - If two threads of the same priority are waiting for the CPU, the scheduler arbitrarily chooses one of them to run.
- Normally, java used preemptive and priority based scheduling algorithm
 - it own library use this.
 - it mean that you can set priority of java thread
 - if higher priority submit to JVM → JVM interrupt the running thread → allow the higher priority to run.
 - The concept of JVM, you have to think about host OS as well.

- If 2 thread has the same priority, JVM use First come First serve.
 - FCFS is non-preemptive

Java Threads Scheduling (Cont.)

- The chosen thread runs until one of the following conditions is true:
 - A higher priority thread becomes runnable.
 - It yields, or its run method exits.
 - On systems that support time-slicing, its time slot has expired.

- you have to understand yields method
- when it said → i am willing to stop using the cpu, allowing some other thread to run.
- if you work with non-preemptive, programmer may need to manage this him/herself.
- window 3.1 is non-preemptive

Java Threads Scheduling (Cont.)

- Priority is in the range of 1-10.
 - The default priority is 5.
 - Method `setPriority()` is used to set a thread priority.
 - `myThread.setPriority(6);`
 - A thread can yield control of the CPU using the `yield()` method.
 - Java can then choose another thread with the same priority to run.
-

For java, higher number → higher priority